

YouTube to Apple Music Intelligent Playback Bridge

Project Report

Functional proof-of-concept for automated cross-platform music playback

Project Type	Desktop automation / browser-extension / media-integration project
Platform	macOS
Primary Browser	Brave Browser
Core Languages	Python and JavaScript
Current Status	Single-track handoff successfully implemented

1. Abstract

This project presents a prototype system that bridges music playback between regular YouTube and Apple Music. A custom Chromium/Brave browser extension detects the currently playing YouTube video and sends its identifier, title, URL, and playback state to a local Python service.

The Python service uses ytmusicapi to identify the corresponding music track and obtain recommendation/radio information from YouTube Music. The track is searched against Apple's music catalog using the iTunes Search API. A suitable Apple Music URL is then sent back through the local bridge.

The browser extension opens the Apple Music track and passes the original YouTube title and artist to an Apple Music content script. That script locates the matching song in Apple Music Web and programmatically clicks its Play button.

YouTube → Extension → Flask → Python → ytmusicapi → Apple Search → Apple Music Web → Automatic Playback

The prototype successfully demonstrates automated single-track handoff without requiring a paid Apple Developer membership. Recommendation-to-queue synchronization remains future work.

2. Problem Statement

Users may discover music through YouTube but prefer listening through Apple Music. Normally they must identify the song, search Apple Music, open the correct result, and start playback manually. The project automates this process.

YouTube song → detect → identify → Apple Music match → open → automatically play

3. Project Objectives

3.1 Primary Objectives

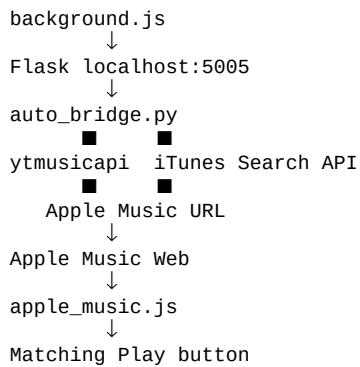
- Detect the currently playing YouTube video.
- Identify title and artist.
- Obtain YouTube Music recommendations.
- Search for the equivalent Apple Music track.
- Prefer the correct regional storefront.
- Open the matched Apple Music track automatically.
- Click the correct Apple Music Play button.
- Minimize manual interaction.

3.2 Long-Term Objectives

- Obtain multiple recommendations.
- Match them against Apple Music.
- Construct an Apple Music queue.
- Preserve the currently playing track.
- Update only future queue entries.
- Eventually provide a native macOS menu-bar application.

4. System Architecture

Regular YouTube / Brave
↓
content.js
↓



5. Technologies Used

Technology	Purpose
Python	Main orchestration and automation language.
Flask	Local HTTP communication service on localhost:5005.
Flask-CORS	Cross-origin support for extension requests.
ytmusicapi	YouTube Music track identification and watch/radio retrieval.
browser.json	Browser-session authentication for ytmusicapi.
iTunes Search API	Apple catalog search and Apple Music URL retrieval.
Brave Browser	Chromium host for YouTube and Apple Music Web.
Manifest V3	Browser extension architecture.
JavaScript	Content scripts, service worker, and DOM automation.
Apple Music Web	Playback target.
AppleScript	Investigated for native Music.app control.
MusicKit / SystemMusicPlayer	Investigated for native Apple Music and queue control.
HTTP / JSON	Local browser-to-Python communication.

6. Development Roadmap Followed

Phase	Milestone	Outcome
1	Concept	Defined the YouTube-to-Apple Music handoff and future queue goal.
2	YouTube recommendation POC	Configured ytmusicapi and successfully retrieved watch/radio tracks.
3	YouTube detection	Built extension to detect video ID, title, URL, and playback state.
4	Local bridge	Built Flask service and auto_bridge.py.
5	Track identification	Used ytmusicapi to identify title and artist.
6	Apple matching	Used iTunes Search API to find Apple catalog matches.
7	Regional URLs	Preferred the India Apple Music storefront.
8	Apple Music Web investigation	Found and tested the DOM Play button.
9	Metadata handoff	Replaced URL query parameters with extension messaging.
10	Single-track completion	Achieved automatic YouTube → Apple Music playback.
Future	Queue synchronization	Match multiple recommendations and maintain future queue entries.
Future	Native macOS app	Potential menu-bar application.

7. Major Successes

- End-to-end automatic YouTube → Apple Music playback works.
- YouTube playback state is detected by the extension.
- Flask reliably connects the extension and Python.
- YouTube tracks are matched to Apple Music despite naming differences.
- India storefront URLs work for the target environment.
- Apple Music Web Play buttons can be located and clicked programmatically.
- The final prototype does not require paid Apple Developer membership.

8. Failure Points and Lessons Learned

Challenge	Resolution / Lesson
MusicKit developer-token failure	Catalog search encountered developer-token authorization limitations; iTunes Search API became the catalog-matching
ApplicationMusicPlayer mismatch	It is intended for app-specific playback rather than controlling Music.app; SystemMusicPlayer is the relevant native API
Native Music.app complexity	AppleScript handled basic playback but was not selected for the main dynamic queue workflow.
Apple Music URL did not auto-play	Opening the track page was insufficient; DOM Play-button automation was required.
URL metadata injection failed	Appending ytTitle/ytArtist to Apple Music URLs caused page-loading problems; extension messaging replaced it.
Extension reload vs page reload	An already-open YouTube tab needed to be reloaded after extension reload so the updated content script was injected
Long Terminal heredocs	Large pasted heredocs repeatedly caused shell input issues; shorter targeted edits were more reliable.
Queue synchronization incomplete	The current prototype handles the current song; dynamic recommendation-to-queue synchronization is future work.

9. Current System Limitations

- ytmusicapi recommendations are not guaranteed to exactly reproduce regular YouTube's personalized Up Next queue.
- Title/artist matching can produce false positives for remixes, covers, live versions, or duplicate releases.
- Apple Music Web DOM selectors may change with website updates.
- Dynamic queue synchronization has not yet been completed.
- The current implementation requires a running local Python service and browser extension.

10. Proposed Future Roadmap

Recommendation extraction

Retrieve a list of YouTube Music watch/radio recommendations.

Batch Apple Music matching

Match each recommendation against Apple's catalog.

Queue construction

Create a queue from matched Apple Music items.

Non-interruptive updates

Preserve the current song while updating only future entries.

Recommendation refresh

Compare new recommendations against the previous list.

Failure recovery

Handle unavailable tracks, loading failures, playback stops, and DOM changes.

Native macOS app

Create a polished Swift/SwiftUI menu-bar front end.

11. Final Technology Stack

Layer	Technology
OS	macOS
Browser	Brave
Extension	Chromium Manifest V3
Extension language	JavaScript
Backend	Python + Flask
YouTube layer	ytmusicapi
Authentication	browser.json
Apple catalog	iTunes Search API
Playback	Apple Music Web
Native research	AppleScript + MusicKit
Transport	HTTP / JSON
Local endpoint	127.0.0.1:5005

12. Project File Structure

```
yt-to-apple/  
  youtube_bridge/  
    venv/  
    browser.json  
    service.py  
    auto_bridge.py  
    music_controller.py  
    music_matcher.py  
    bridge_match_test.py  
    poc.py  
    extension/  
      manifest.json  
      background.js  
      content.js  
      apple_music.js
```

browser.json contains authenticated session information and must never be committed to a public repository.

```
browser.json  
oauth.json  
venv/  
.env
```

13. Security Considerations

- Treat browser.json as sensitive authentication material.
- Never upload browser.json to GitHub or share it publicly.
- Regenerate authentication data if it is exposed.
- Keep Flask bound to 127.0.0.1 unless remote access is deliberately required.
- Do not expose the local command endpoints to the public network.

The current Flask service binds to 127.0.0.1:5005, limiting access to the local machine.

14. Evaluation

Component	Status
YouTube Detection	Completed
YouTube Track Identification	Completed
Apple Music Matching	Completed
Regional URL Handling	Completed
Apple Music Opening	Completed
Automatic Playback	Completed
Extension ↔ Python Communication	Completed
YouTube Recommendations	Prototype
Apple Music Queue	Future Work
Native macOS Application	Future Work

15. Conclusion

The YouTube to Apple Music Intelligent Playback Bridge successfully progressed from a concept to a working end-to-end prototype. The system detects a YouTube song, identifies its title and artist, finds a corresponding Apple Music catalog entry, opens the Apple Music track, and automatically starts playback.

Several technical obstacles were encountered. MusicKit catalog access was limited by developer-token requirements, ApplicationMusicPlayer was unsuitable for controlling Music.app state, direct Apple Music URLs did not consistently initiate playback, and passing metadata through URL query parameters caused page-loading problems.

Each failure led to a more suitable architecture. The final prototype uses ytmusicapi, Flask, a Chromium extension, the iTunes Search API, and Apple Music Web automation. This provides a strong foundation for the next stage: converting YouTube Music recommendations into an automatically maintained Apple Music queue.

Current achievement:

YouTube → detect → identify → Apple search → match → open → find Play button → playback

The project should therefore be presented as a functional proof-of-concept for automated YouTube-to-Apple-Music intelligent playback. Recommendation-to-queue synchronization is the principal future engineering objective.

References and Technical Documentation

- ytmusicapi documentation — browser authentication and API reference.
- Chrome Extensions documentation — Manifest V3, messaging, Tabs API, and content scripts.
- Apple Developer Documentation — MusicKit, SystemMusicPlayer, and MusicPlayer.Queue.
- Apple Support — Apple Music on the web.
- Apple iTunes Search API — music catalog search endpoint.